Interactiediagrammen

1 Inleiding

Dit document hoort bij de cursus Design Patterns. Het behandelt interactiediagrammen.

Interactiediagrammen zijn UML-diagrammen, waarmee je modelleert hoe objecten met elkaar samenwerken. Interactiediagrammen zijn dus niet bedoeld om structuur te tonen, maar om *gedrag* te tonen.

Er zijn twee soorten interactiediagrammen: sequentiediagrammen en collaboratiediagrammen. In dit document wordt uitgelegd wat sequentie- en collaboratiediagrammen zijn, wat hun bedoeling is, en hoe je ze tekent.

Interactiediagrammen worden hier niet ten gronde uitgelegd. Er is met sequentie- en collaboratie-diagrammen meer mogelijk dan in dit document aan bod komt.

2 Wat zijn interactiediagrammen?

In de cursus Analysis & Design werden al een aantal diagramtechnieken behandeld. De statische structuur van een applicatie modelleren we met een klassendiagram, de toestandsveranderingen van een object met een toestandsdiagram, de functionele eisen met een use-cases-diagram, de workflow van een organisatie met een activiteitendiagram, de softwarearchitectuur met een component-diagram en de hardwarearchitectuur met een opstellingsdiagram.

Een klassendiagram modelleert de statische structuur van een applicatie, dus welke objecten er zijn, en welke relaties die objecten met elkaar hebben. Maar objecten zijn geen passieve, onveranderlijke dingen. In een progamma werken ze met elkaar samen. Objecten creëren andere objecten, en gebruiken de methodes van andere objecten. In het jargon: ze sturen boodschappen naar elkaar (messages in het Engels). Hoe die samenwerking verloopt kunnen we tonen met *interactie-diagrammen*.

Interactiediagrammen zijn nuttig op verschillende niveaus.
Op een hoog niveau tonen ze hoe verschillende systemen of subsystemen met elkaar samenwerken. Bijvoorbeeld: hoe een kassasysteem met een inventarissysteem samenwerkt om bij elke verkoop de inventaris up-to-date te houden.
Op detailniveau tonen ze hoe concrete objecten (dus instanties van klassen) met elkaar samenwerken.

Niet vergeten

De term 'object' heeft in dit hoofdstuk een ruimere betekenis dan die van 'instantie van een klasse'. Het kan ook een instantie van een subsysteem zijn.

3 Boodschappen

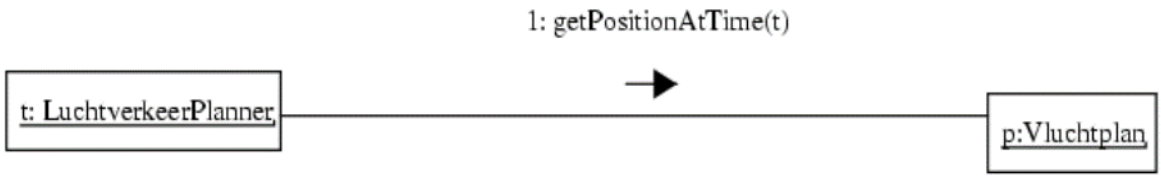
Bij het uitvoeren van acties werken objecten samen. Zij sturen *boodschappen* naar elkaar (in het Engels: *messages*). Wanneer een object A een bepaalde dienst van een object B nodig heeft om zijn taak uit te voeren, dan zal A een boodschap sturen naar B. Dat wil zeggen: A gebruikt de API van B, A roept methodes van B aan.

In het voorbeeld in figuur 1 stuurt het object `t` een boodschap naar het object `p`. De boodschap heeft de naam `getPositionAtTime`, en het heeft een parameter `t`. In code (bijvoorbeeld in Java) zou dat er allicht zo uit zien:

```
position = p.getPositionAtTime(currentTime);
```

(`position` en `currentTime` zijn dan variabelen van het programma.)

Een boodschap in een interactiediagram komt dus overeen met een methodeaanroep of een API-aanroep in code. De boodschap vertrekt bij de caller (object t in het voorbeeld), en komt aan bij de callee (object p).



Figuur 1: boodschap versturen

Niet vergeten

Hoe zie je dat er in het diagram objecten staan, en geen klassen?

- In de rechthoeken staat tekst met een dubbelepunt. Objecten hebben een naam (het stuk voor de dubbelepunt) en een type (het stuk na de dubbelepunt). Klassen kunnen enkel een type hebben, dus daar geen dubbelepunt.

  “p:Vluchtplan” is een object van type Vluchtplan, met de naam p.

  “:Vluchtplan” is ook een object van type Vluchtplan, maar dan zonder naam.

  “Vluchtplan” is een klasse.

- Naam en type zijn onderstreept.

  Onderstrepen gebeurt niet altijd (zie bijvoorbeeld de meeste figuren in dit hoofdstuk). Sommige softwarepakketten laten de onderstreping weg (bijvoorbeeld: Visual Paradigm). Als je een diagram met de hand schetst wordt de onderstreping ook dikwijls weggelaten.

Bij het modelleren van scenario's kunnen we de nadruk leggen op twee verschillende zaken:

- We kunnen ons concentreren op het tijdsverloop, op de opeenvolging van boodschappen in de tijd. In dat geval gebruiken we een *sequentiediagram (sequence diagram)*.
- We kunnen ons ook concentreren op het structurele aspect, op de relaties tussen de objecten die samenwerken. In dat geval gebruiken we een *collaboratiediagram (collaboration diagram)*.

Sequentiediagrammen en collaboratiediagrammen zijn de twee soorten interactiediagrammen. Van die twee soorten worden sequentiediagrammen veruit het meest gebruikt.

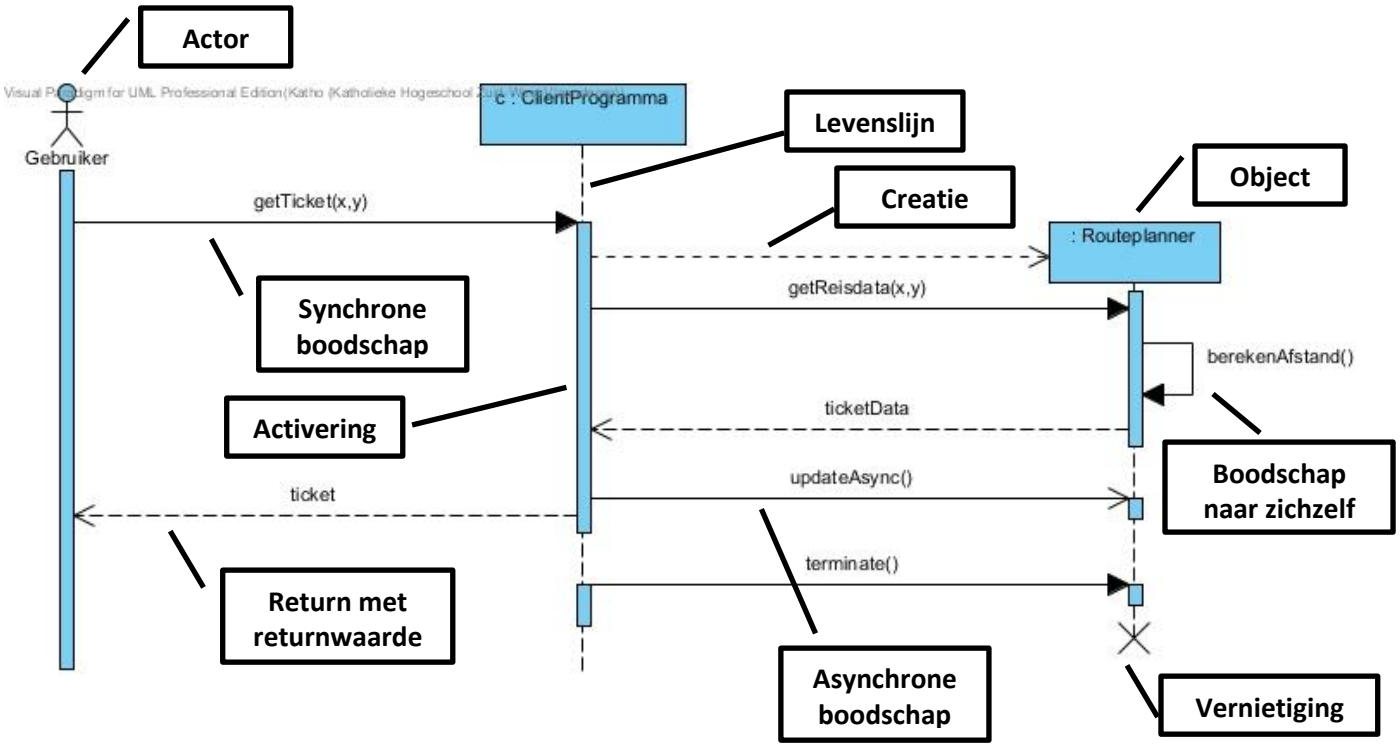
4 Sequentiediagrammen

4.1 Doel

Een sequentiediagram toont de samenwerking tussen objecten, waarbij de nadruk ligt op het tijdsverloop. Een sequentiediagram is dus heel geschikt om te tonen in welke volgorde de deelnemers in een interactie boodschappen sturen naar elkaar.

4.2 Basiselementen

Laten we de belangrijkste componenten van een sequentiediagram eens bekijken. Ze worden ook aangeduid op figuur 2.



Figuur 2: elementen van een sequentiediagram

Tijd

De tijd staat niet expliciet aangegeven op een sequentiediagram.

In een sequentiediagram loopt de tijd van boven naar onder. Wat hoger staat gebeurt eerst, wat lager staat gebeurt later.

Voorbeeld: in figuur 2 is het eerste wat gebeurt dat de actor Gebruiker een boodschap getTicket stuurt naar het object c:ClientProgramma. Het volgende is de creatie van een object :Routeplanner door het object c:ClientProgramma. Daarna stuurt c:ClientProgramma de boodschap getReisdata naar :Routeplanner.

Object

Een object wordt aangeduid in een rechthoek.

Voorbeeld: c:ClientProgramma is een object van het type ClientProgramma, met de naam c.

Wanneer we geen specifieke naam aan het object willen geven, dan laten we de naam gewoon weg. We schrijven dan enkel de klassenaam, voorafgegaan door een dubbelepunt.

Voorbeeld: :Routeplanner is een anoniem object van het type Routeplanner.

Actor

De start van een sequentie kan gegeven worden door een externe actor. We stellen een actor op dezelfde manier voor als in use-casesdiagrammen: een 'stickman' met de naam van de actor er onder.

Niet alle sequentiediagrammen hebben een actor.

Levenslijn

De levenslijn is een lijn die onder een object hangt. Elk object en elke actor heeft een levenslijn.

Het uitzicht van de lijn toont de toestand van het object. Een stippellijn geeft aan dat het object bestaat, maar niet actief is. Een dikke balklijn geeft aan dat het object actief betrokken is bij de sequentie (zie volgende item: activering).

Boodschappen vertrekken van een levenslijn en komen aan op een levenslijn.

Activering

Wanneer een object een boodschap krijgt, wordt het actief. De boodschap zorgt er voor dat één van de operaties van het object uitgevoerd wordt.

De activering van een object duiden we aan met een balkje dat iets dikker is dan de levenslijn. De activering begint bij de operatieaanroep en eindigt wanneer de operatie ten einde is (en eventueel een returnwaarde teruggeeft).

Wanneer je een sequentiediagram tekent met software, dan zorgt de software voor het goede uitzicht van de levenslijnen. Wanneer je een sequentiediagram schetst op papier, dan kun je de balklijn eventueel vervangen door een dikke volle lijn.

Boodschap (message)

Een boodschap is een operatieaanroep, een verzoek aan een object om iets uit te voeren of om informatie te geven.

Een boodschap is een horizontale pijl. De pijl wijst naar het object dat de boodschap krijgt. De pijl vertrekt bij het object dat de boodschap stuurt.
Boven de pijl schrijf je de naam van de boodschap. Eventueel kun je de naam laten volgen door de lijst met parameters tussen haakjes. Anders schrijf je enkel de haakjes.

Voorbeeld: in figuur 2 is getReisdata() een boodschap die door c:ClientProgramma naar het object :Routeplanner gestuurd wordt. In softwaretermen: Clientprogramma roept de methode getReisdata() van Routeplanner aan.
De boodschap heeft twee parameters: x en y.

Een *synchrone boodschap* (d.w.z. een boodschap waarbij het aanroepende object wacht om verder te gaan tot de aangeroepen operatie is uitgevoerd) duiden we aan met een pijl met een gesloten pijlpunt.
Methode-aanroepen in Java zijn een voorbeeld van synchrone boodschappen.

Een *asynchrone boodschap* (d.w.z. een boodschap waarbij het aanroepende object niet wacht op het einde van de operatie) duiden we aan met een pijl met een open pijlpunt.

Voorbeeld: in figuur 2 is getReisdata() een synchrone boodschap, en updateAsync() een asynchrone boodschap.

Creatieboodschap

Een constructor of creatieboodschap duiden we aan met een pijl in streeplijn die wijst naar het object dat gecreëerd wordt.

Een alternatief is om de creatie te tekenen met een gewone boodschap met de naam <<create>>.

Boodschap naar zichzelf

Een object kan ook een boodschap naar zichzelf sturen. We kunnen de activeringsbalk dan verdubbelen, maar dat hoeft niet.

Vernietiging

Wanneer een object vernietigd wordt, dan beëindigen we de levenslijn onderaan met een kruis.

Zo een kruis wordt enkel getekend in het geval van een expliciete vernietiging, dus als in het programma instructies staan om het object te vernietigen. Voor automatische vernietiging door de garbage collector teken je geen kruis.

Return

Operaties geven vaak een returnwaarde. Een return duiden we aan met een stippelpijl.
Return mag je weglaten als dat het diagram beter leesbaar maakt.

Je kunt de returnwaarde bij een return aangeven, maar ook dat is niet verplicht.

Niet vergeten

Je hebt bij het tekenen van een sequentiediagram heel wat vrijheid over de mate van detaillering.
Hoeveel detail is nodig? Dat is een kwestie van gezond verstand. Het doel van het diagram is om te tonen hoe softwareobjecten met elkaar samenwerken. Als het diagram die boodschap duidelijk overbrengt, dan is het goed.

Wanneer je diagrammen met software tekent (bijvoorbeeld Visual Paradigm), dan helpt die software om alles volgens de standaard, overzichtelijk en leesbaar te maken. In zo een diagram kun je dus gemakkelijker details opnemen.

Wanneer je diagrammen met de hand op papier of op een bord tekent, dan ziet alles er een stuk minder net uit. Het is dan dikwijls goed om details weg te laten, zodat het diagram overzichtelijk blijft.

4.3 Oefening

Nieuw telefoonnummer

Teken het volgende scenario met een sequentiediagram:

Een object van de klasse `ContactControl` krijgt een message (van een onbekende bron):
`setTelnr('Pieters Jan','056805612').`

Dit betekent dat het telefoonnummer van de klant "Pieters Jan" gewijzigd moet worden en de waarde "056805612" moet krijgen.

Daarvoor creëert `ContactControl` een object van de klasse `ConctactDb`. Aan dit object stuurt het de boodschap
`getContact('Pieters Jan')`.

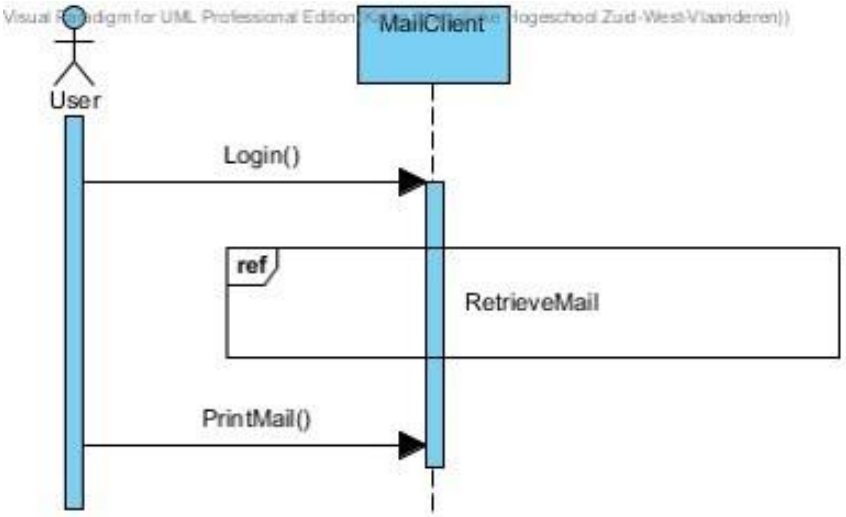
`ContactControl` krijgt van ContactDb een object terug van de klasse `Contact`. Die klasse `Contact` heeft een operatie
`setTelefoonnummer(telnr: String)`

Die operatie wordt uitgevoerd. Daarna wordt bij `ContactDb` de operatie
`storeContact(c: Contact)`
aangeroepen om Jan Pieters weer op te slaan.

Teken een sequentiediagram dat met dit scenario overeenkomt.

4.4 Verwijzing, tijdsbeperking, alternatieven, iteratie

Sequentiediagrammen kunnen **verwijzen** naar andere diagrammen, waar een deel van de interactie meer in detail uitgewerkt wordt. Een dergelijke verwijzing wordt getekend als een *frame*. Een frame is een rechthoek met een label in de linkerbovenhoek. Het label staat in een rechthoekje met een afgeknipte rechteronderhoek. Voor een verwijzing is het label 'ref'. De naam van het diagram waarnaar verwezen wordt staat in het frame. (Voorbeeld: figuur 3)



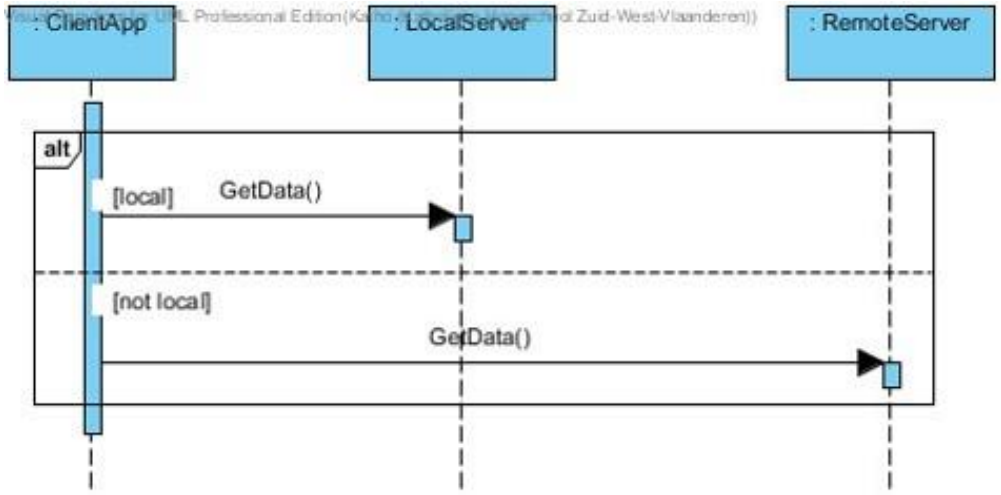
Figuur 3: verwijzing naar een ander sequentiediagram

Tijdsbeperkingen kun je aangeven als tekst tussen accolades. (Voorbeeld: figuur 4. De tijd tussen de boodschappen EnterTransaction en ConfirmTransaction mag hoogstens 300 seconden zijn.)

Figuur 4: tijdsbeperking

Soms heeft een scenario 2 **alternatieve verlopen**. Dat kun je modelleren met een frame met het label 'alt'. Het frame is met horizontale stippellijnen onderverdeeld in twee of meer delen. Elk deel is een alternatief verloop van de sequentie. De voorwaarde om voor het ene of het andere alternatief te kiezen (de 'guards') moet je schrijven als tekst tussen vierkante haken.

Figuur 5 toont een voorbeeld: :ClientApp kan ofwel GetData() aanroepen op een lokale server, ofwel GetData() aanroepen op een niet-lokale server. De guards ([local], [not local]) geven aan wanneer welk alternatief van toepassing is.

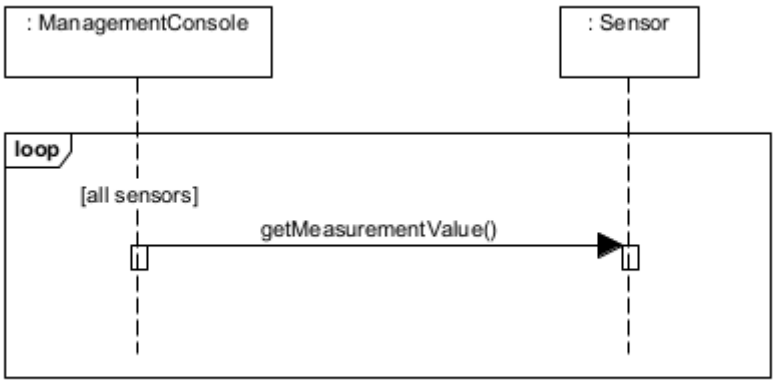


Figuur 5: alternatieve verlopen

In een scenario kan het soms gaan over een **iteratie** over een reeks van gelijkaardige objecten. Dat modelleer je met een frame met het label 'loop'. In een guard schrijf je dan waarover er geïtereerd wordt.

Figuur 6 toont een voorbeeld: de ManagementConsole stuurt de boodschap GetMeasurementValue() niet naar 1 Sensor, maar naar alle leden van een collectie Sensor-objecten. In pseudocode:

```
for (sensor: Sensors) { sensor.GetMeasurementValue(); }
```



Figuur 6: een frame met een iteratie

Extra informatie

Niet alles wat je met sequentiediagrammen kunt doen is hier aan bod gekomen. Meer uitgebreide informatie kun je vinden in de literatuur (bijvoorbeeld: (Warmer & Kleppe, 2011)), of op het web (zoekterm: 'UML sequence diagram').

5 Collaboratiediagrammen

Een andere naam voor collaboratiediagram is *communicatiediagram*.

5.1 Doel

Een sequentiediagram toont de samenwerking tussen objecten, waarbij de nadruk ligt op de relaties tussen de verschillende objecten.

5.2 Elementen

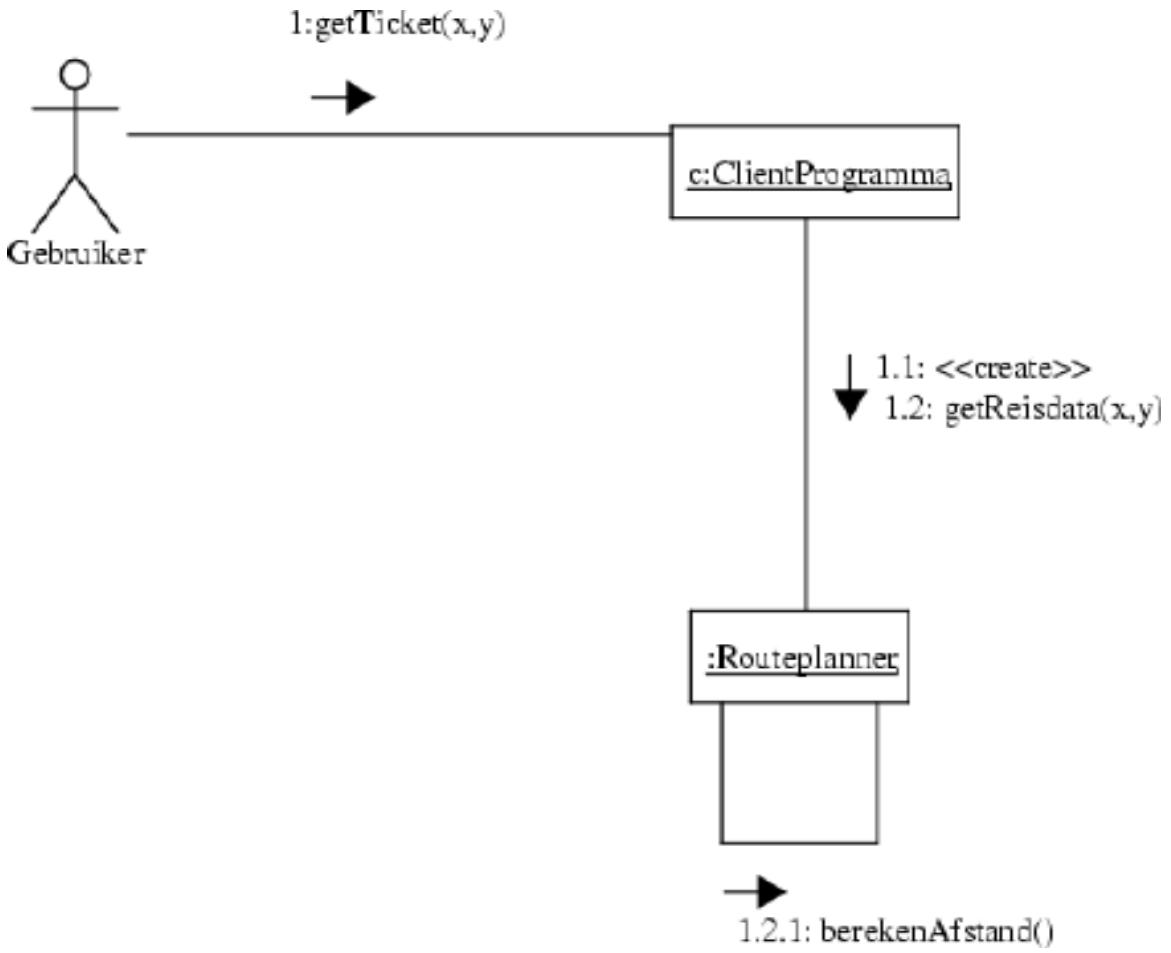
Een collaboratiediagram lijkt wat op een klassendiagram: rechthoeken verbonden met lijnen. Maar een klassendiagram toont klassen, en een collaboratiediagram toont objecten.

De boodschappen worden geschreven naast de associatielijnen, met een kort pijltje om duidelijk te maken welk object de boodschap stuurt en welk object de boodschap ontvangt.

De volgorde van boodschappen wordt aangegeven door ze te nummeren. Er zijn twee stijlen van nummeren mogelijk: ofwel doorlopend (1, 2, 3, …), ofwel met niveaus (1, 1.1, 1.2, 1.2.1, 1.3, 2, …). De tweede stijl benadrukt de hiërarchie van functieaanroepen. Als boodschap 1 een object A activeert, dan is boodschap 1.1 de eerste boodschap die A als reactie aanroept op een ander object. Je bent vrij om de stijl te kiezen die je zelf het duidelijkste vindt.

Figuur 7 toont een voorbeeld. Het is een stuk van de interactie uit figuur 2 (de boodschappen updateAsync() en terminate() ontbreken hier).

Returnboodschappen zijn hier weggelaten: ze zouden de figuur te overladen maken.



Figuur 7: collaboratiediagram

De meeste case-tools kunnen sequentiediagrammen omzetten naar collaboratiediagrammen, en omgekeerd.

5.3 Oefening

Nieuw telefoonnummer

Zet de oefening uit 4.3 om naar een collaboratiediagram.

Een voorbeeldoplossing vind je op Toledo.

6 Bibliografie

Larman, C. (2005). *Applying UML and Patterns.* Pearson.

Warmer, J., & Kleppe, A. (2011). *Praktisch UML.* Pearson Education Benelux.